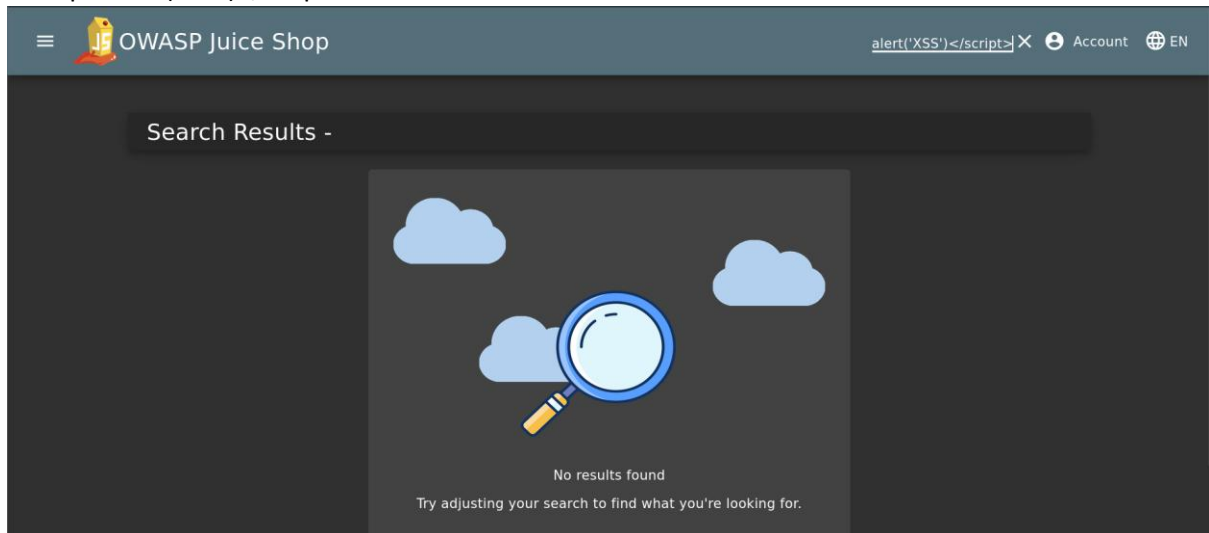
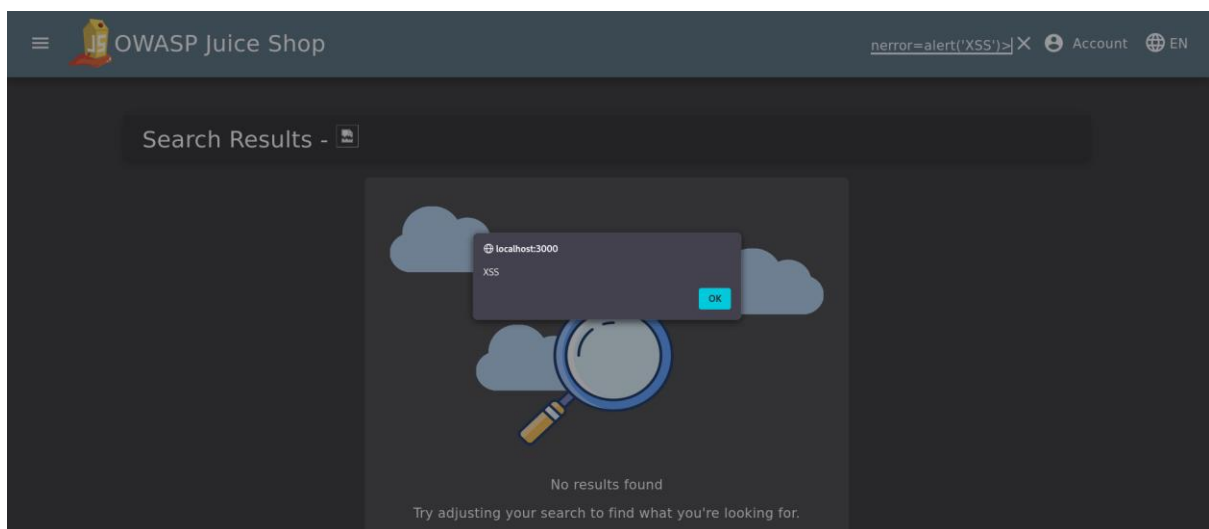


```
<script>alert('XSS')</script>
```



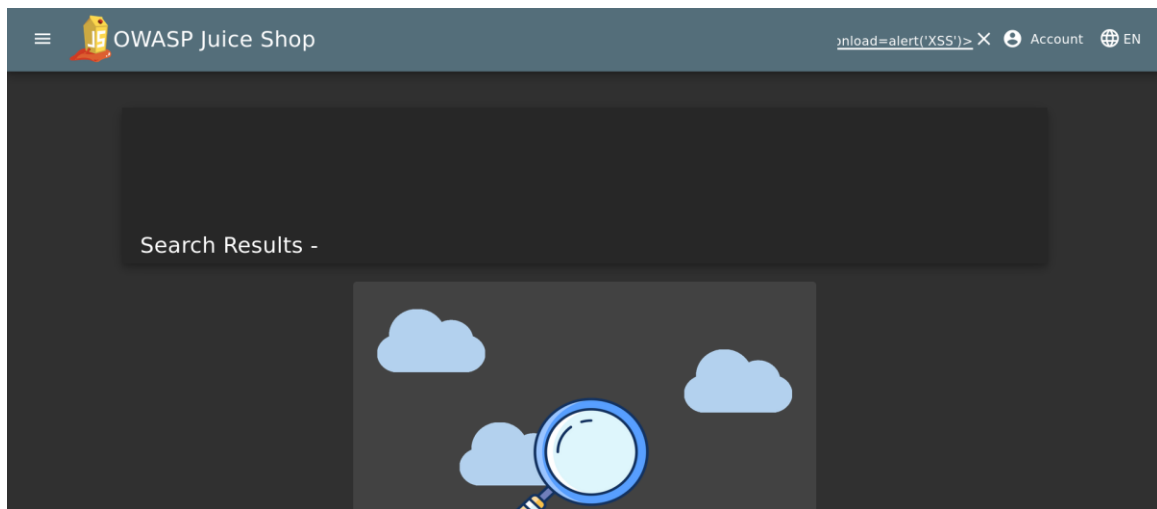
This payload tries to inject a script tag that runs JavaScript. If successful, it shows a popup alert with the message XSS. In many cases, this may be blocked because modern applications filter script tags.

```
<img src=x onerror=alert('XSS')>
```



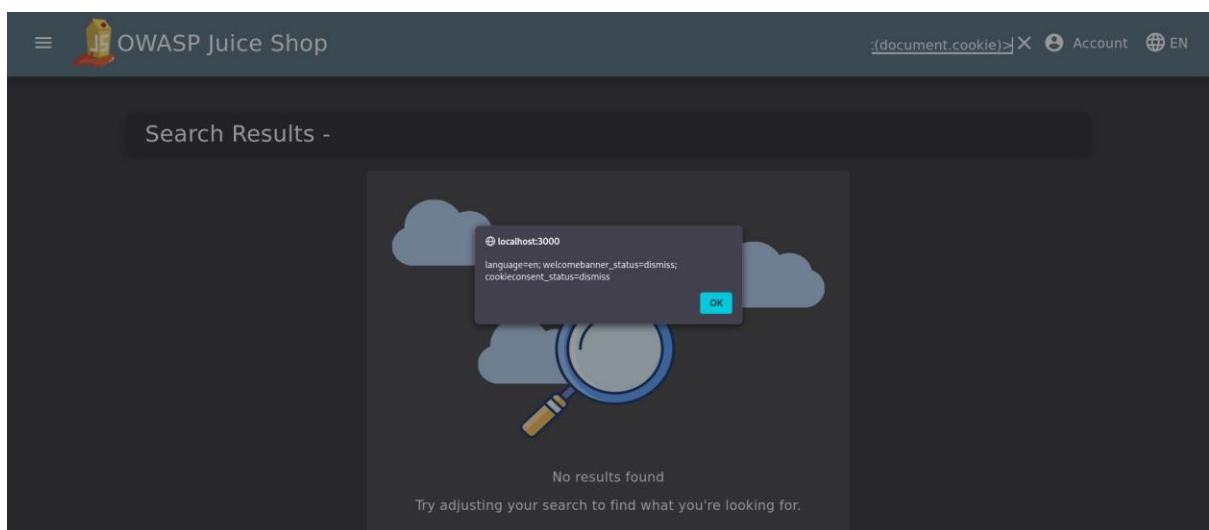
This creates an image with an invalid source. Since the image fails to load, the onerror event runs JavaScript and shows an alert. This often works even when script tags are blocked.

```
<svg/onload=alert('XSS')>
```



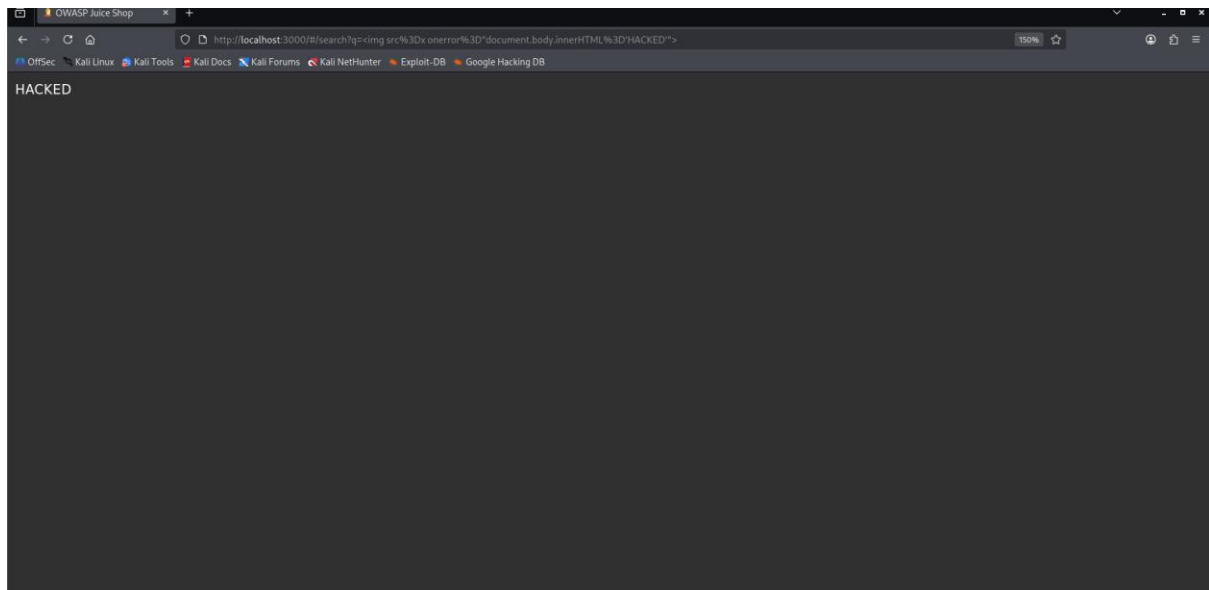
This uses an SVG element and triggers JavaScript when it loads. It is another way to bypass simple filters and execute code.

```
<img src=x onerror=alert(document.cookie)>
```



This payload shows the cookies stored in the browser. If it works, it demonstrates how attackers can access sensitive information like session cookies.

```
<img src=x onerror="document.body.innerHTML='HACKED'">
```



This modifies the entire webpage content and replaces it with the word HACKED. It shows how attackers can change the structure of a webpage.

The image tag payload worked best because it bypasses basic filtering. Applications often block script tags but may not properly handle event attributes like onerror, making this payload more effective.